

Stemz HR Assistant

Speed Improvements + Confidential Data to HR Escalation Report

Generated: 2026-09-02 | Scope: chatbot backend flow, RAG guardrails, Ask-HR email path

1. Executive Summary

This report documents two key tracks:

- **Faster chat response behavior:** model routing, context trimming, timeout/caching, fallback answers, and user-visible streaming statuses.
- **Safer handling of confidential policy content:** deterministic refusal before LLM generation and escalation support through the Ask-HR channel.

Net result: lower latency and better resilience, while preventing accidental disclosure of protected HR content.

2. What Was Improved for Speed

Area	Implementation	Impact
Model selection	Speed-first routing with confidence-based escalation	Fast model serves most requests, slower/heavier model only when needed
Context size	Prior chat window is trimmed by count and character cap	Lower prompt size, lower token cost, faster generation
Timeout control	LLM timeout enforced for streaming calls	Prevents long-tail hangs
Prompt caching hints	Cache-control markers on stable system/prior blocks	Improves repeated multi-turn performance
Fallback answer	Local short answer from top retrieved chunk when LLM fails	User receives usable output instead of hard failure
Perceived performance	Early SSE status frames (searching/composing)	Immediate UX feedback while backend is working

2.1 Before vs After: Model Routing

Before (single default model):

```
selected_model = settings.CLAUDE_MODEL_DEFAULT
```

After (speed-first + escalation):

```
selected_model = _pick_model(top_score)
```

```
def _pick_model(top_score):
    if settings.FORCE_FAST_MODEL:
        return settings.CLAUDE_MODEL_FAST
    if settings.SPEED_FIRST_MODE:
        if top_score < settings.ESCALATION_SCORE_THRESHOLD:
            return settings.CLAUDE_MODEL_ESCALATION
        return settings.CLAUDE_MODEL_FAST
    if top_score < settings.ESCALATION_SCORE_THRESHOLD:
        return settings.CLAUDE_MODEL_ESCALATION
    if top_score >= settings.FAST_SCORE_THRESHOLD:
        return settings.CLAUDE_MODEL_FAST
    return settings.CLAUDE_MODEL_DEFAULT
```

2.2 Before vs After: Prompt Window Trimming

Before:

```
prior_llm_messages = [{'role': m.role, 'content': m.content} for m in
prior_messages]
```

After:

```
prior_llm_messages = _trim_prior_llm_messages(prior_llm_messages)

def _trim_prior_llm_messages(messages):
    max_messages = max(0, settings.MAX_HISTORY_MESSAGES)
    max_chars = max(1, settings.MAX_HISTORY_CHARS)
    ...
```

2.3 Before vs After: Timeout + Fallback

Before:

```
client = Anthropic(api_key=settings.ANTHROPIC_API_KEY)
for token in stream_answer(...):
    yield token
```

After:

```
client = Anthropic(
    api_key=settings.ANTHROPIC_API_KEY,
    timeout=settings.LLM_TIMEOUT_S
)

try:
    for token in stream_answer(...):
        yield token
except Exception:
    fallback = _fallback_answer_from_retrieved(retrieved)
    yield fallback
```

3. Confidential Data Handling and HR Escalation

Layer	What happens	Security effect
Ingestion	Markdown frontmatter sets <code>confidential: true</code> and parser stamps chunk flag	Confidentiality is machine-enforced per chunk
Guardrail	Retrieved set checked for confidential hit at relevant score	Sensitive requests refused deterministically
Generation gate	Refusal path exits before LLM call	Confidential text never enters model prompt
Defense in depth	Any remaining confidential chunks are filtered out pre-generation	Secondary leakage prevention
Ask-HR endpoint	Escalation request persisted, then emailed to HR recipients	No data loss if email delivery fails

3.1 Before vs After: Confidential Guard

Before:

```
# Retrieved chunks flowed directly to generation
```

After:

```
if confidential_hit(retrieved):
    yield _sse({'type': 'token', 'text': CONFIDENTIAL_REFUSAL_MESSAGE})
    Message.objects.create(..., content=CONFIDENTIAL_REFUSAL_MESSAGE, ...)
    yield _sse({'type': 'done', 'confidence': 'refused', 'escalate': True})
    return
```

3.2 Before vs After: Defense in Depth Filter

Before:

```
# No explicit confidential filter before passing chunks to LLM
```

After:

```
retrieved = [r for r in retrieved if not is_confidential(r['chunk'])]
```

3.3 Before vs After: Ask-HR Reliability

Before:

```
# Escalation path depended mainly on successful email delivery
```

After:

```
record = AskHRRequest.objects.create(...)

try:
    send_mail(...)
    record.email_status = 'sent'
except Exception as exc:
    record.email_status = 'failed'
    record.email_error = str(exc)[:2000]
    # Still persisted in DB for HR follow-up
```

4. End-to-End Flow

Step	Condition	Outcome
1. User asks question	Input is valid and within limits	Continue
2. Retrieve	Top chunks returned with scores	Continue
3. Confidential check	Any relevant confidential chunk present	Refuse and suggest HR contact
4. Refusal check	Low confidence / out of scope	Generic refusal and escalate option
5. Generate	Non-confidential grounded context available	Stream answer with citation/source
6. Ask-HR	User escalates	Persist request + email HR recipients

5. Validation Checklist

- Ask questions from non-confidential policies: answer should stream normally.
- Ask questions targeting confidential policies: confidential refusal should trigger.
- Force model timeout or failure: fallback answer should appear.
- Use Ask-HR with valid email: request should persist and send.
- Simulate mail failure: request should remain saved with failed status.

Operational note: For any newly confidential document, set frontmatter to `confidential: true` and re-run ingestion so chunk metadata is refreshed.

6. Key Code Locations

- Speed routing and stream flow: `apps/chatbot/views.py`
- Prompt build and timeout/caching: `apps/rag/services/generate.py`
- Confidential guardrail logic: `apps/rag/services/guardrail.py`

- Frontmatter-to-chunk confidential mapping: apps/knowledge/services/parser.py
- Ask-HR persistence model: apps/chatbot/models.py
- Speed/confidence/settings knobs: config/settings/base.py

